PDF Merger & Splitter Suite

Technical Design & Operations Documentation

Author:	brithvik8
Repository:	https://github.com/brithvik8/Mini_Python_projects
Branding Accent Color:	Purple (#463671)
Created Date:	August 2026

1. Technology Stack & Decoupled Design

This utility desktop application is engineered in Python utilizing the Model-View-Controller/Presenter design pattern. It decouples the core operations from the user interface, making it easily testable, modular, and scalable.

1.1 Core Tools & Libraries

Tool	Function & Rationale
Python 3.12+	Core programming language utilizing OOP structures.
CustomTkinter	Modern GUI library providing light and dark appearance settings and rounded widgets.
pypdf	Pure-python dependency used to read, manipulate, rotate and write PDF files.
threading	Core threading library used to offload PDF merges and extracts to background processes.
pytest	Automated testing framework for verification checks.

2. Merging Algorithm Working Principle

The PDF merging engine takes a list of absolute file paths and compiles them into a single target file. The algorithm follows these steps:

1. **Validation**: Verifies that the queue contains at least 2 file paths. If not, it raises a *ValueError*.
2. **Initialization**: Creates an instance of *pypdf.PdfWriter*.
3. **Looping**: For each file path, it validates the file's existence. It then initializes a *pypdf.PdfReader* and appends its contents using *writer.append(reader)*.
4. **Directory Creation**: Dynamically verifies if the output path directory exists, creating it if necessary.
5. **Write Out**: Opens the destination file in binary write mode ('wb') and saves the compiled pages. The writer is then closed to free resource handles.

Code Snippet (Merge Logic):

```
def merge_pdfs(self, file_paths: list[str], output_path: str) -> None:
    writer = PdfWriter()
    try:
        for path in file_paths:
            reader = PdfReader(path)
            writer.append(reader)
        with open(output_path, "wb") as output_file:
            writer.write(output_file)
    finally:
        writer.close()
```

3. Page Selection & Splitting Algorithms

Extracting specific sections requires transforming user-facing, 1-indexed range selections (e.g. '1-3, 5') into zero-indexed Python list ranges.

3.1 Range Parsing Algorithm

The string parsing function *parse_page_ranges* operates as follows:
1. **Splicing**: Splits the string input by comma separators (',').
2. **Sub-ranges Check**: Inspects each slice for a hyphen ('-'). If present, it extracts the start and end values.
3. **Type Validation**: Conversions to integers are protected by try-except blocks. Non-integers raise a descriptive error.
4. **Bounds Verification**: Checks that page indices are greater than zero, start <= end, and end <= total pages in the source PDF.
5. **Index Mapping**: Converts values to 0-based index structures and returns a sorted list of unique page indexes.

3.2 Document Extraction & Bursting

The extraction writes the pages at the resolved indices to a new *PdfWriter* instance. For the **Split All Pages** operation, the engine loops through the document and outputs each single page into its own target path using a filename formatting scheme: *{original_name}_page_{index}.pdf*.

4. Multithreading & GUI Response Loop

Running heavy file operations on CustomTkinter's main event thread causes the desktop window to freeze (i.e. 'Not Responding'). To maintain a fluid UI, the application implements active multi-threading:

1. The trigger button initiates a background thread using the built-in *threading.Thread* class with *daemon=True*.
2. The UI button enters a disabled state, and a progress bar starts displaying progress increments.
3. The background thread carries out the PDF operation. Once completed, standard thread-safe message boxes are called to present status warnings or success messages, and the UI buttons are re-enabled.